

Table for Four: Retrieval (RAG) Submission

CMU AI Agent Certification Capstone • Week 3 Author: Manish Bhatt

Table for Four is a dining concierge. A guest describes the evening they want in plain language, and the agent (Dino) finds suitable restaurants, points out any special offers that apply, and books the table.

1. Is retrieval required? Yes, for one part of the agent.

Alongside each recommendation, the concierge tells the guest about **perks**: special offers a restaurant is running, written as short descriptions such as *"a complimentary gluten-free tasting flight from our fully celiac-safe kitchen, ideal for a birthday."* Offers like these need retrieval for three reasons.

- **The model has never seen them.** The offers belong to us, not to the public internet. Without retrieval the model can only guess, and a guessed discount is one the guest discovers is fake at the restaurant.
- **Matching them is about meaning, not words.** A guest saying *"somewhere safe for a friend with celiac, and it's her birthday"* should find the offer above, even though almost none of the words line up. Searching for exact words misses it; searching by meaning does not.
- **They change.** Offers expire and are limited to certain days and group sizes, so they have to live somewhere that can be updated, rather than being written into the agent's instructions.

Just as important, retrieval is **not** used elsewhere in the agent. Finding restaurants is an ordinary search against a mapping service, and a reservation is a record in a database. Both are exact lookups with a right answer, and putting a meaning-based search in front of them would only add vagueness. Retrieval is used in the one place where meaning genuinely matters.

2. How the retrieval works

The offers are kept in a separate store, outside the model, holding roughly two dozen offers across ten restaurants. Each offer is converted into a numerical representation of its meaning, so that descriptions that mean similar things sit near each other, and a guest's request can be compared against all of them at once. This conversion runs locally on the machine, so the feature needs no external service, costs nothing per request, and works offline.

When the guest's request is understood, the agent searches the offer store and combines two kinds of judgement:

1. **Meaning.** How closely the offer's description matches what the guest actually asked for.
2. **Eligibility.** Whether the offer is still valid, available on the requested day, allowed for the size of the party, and attached to a restaurant that is actually on the shortlist.

Offers that fail on eligibility are discarded outright. The rest are ranked with meaning weighted more heavily than eligibility, which is deliberate: eligibility decides whether an offer *can* be used, but meaning decides whether it is the *right* one to mention. The best remaining offer is attached to each restaurant before the agent writes its recommendation.

3. Retrieval visibly changes what the agent says

Take the request *"a birthday dinner for four this Friday, one guest is gluten-free."* Three offers survive the eligibility checks:

RANK	RESTAURANT	OFFER	WHY IT RANKED HERE
1	Nonna's Gluten-Free Kitchen	Gluten-Free Tasting Flight	Strongest match to the actual request
2	Osteria Midtown	Weekend Family Feast	A perfect fit on paper: a group offer, valid Friday
3	Verdant	Plant-Based Chef's Tasting	Loosely related; a fallback at best

Without retrieval, the agent could only say something vague, such as *"ask the host whether they're running any specials"*, or invent an offer outright. **With retrieval**, it can say: *"Nonna's has a complimentary gluten-free tasting flight, and their kitchen is fully celiac-safe."* Both the offer and the reassurance about the kitchen come from the retrieved description rather than from the model's imagination.

The example also shows retrieval changing *which* restaurant is recommended. The second option is a flawless fit on the mechanical criteria: it is a group offer, it is valid on a Friday, and it suits a party of four. Judged on those criteria alone it would come first. It only loses because the first option is a far better match to what the guest actually cares about, which is the gluten-free birthday. The retrieved material, not the wording of the prompt, decides the answer.

4. Key design choices

- **Where the offers come from.** There is no dataset of real restaurant offers that can be used freely, so the catalogue is a realistic set we authored ourselves. Every offer the agent surfaces is labelled with its origin, so it is never passed off as more real than it is.
- **How the material is divided up.** Each offer is stored whole, as a single short entry, rather than being cut into fragments. An offer description is only a sentence or two and is already a complete thought, so splitting it would break it apart for no gain. The description itself is what gets searched by meaning; the practical details attached to it (which restaurant, minimum party size, valid days, expiry date) are kept as separate labelled fields, so they can be checked exactly rather than being guessed at from the wording.
- **How many results.** The store deliberately returns more candidates than needed, because the eligibility checks happen afterwards and would otherwise leave the agent empty-handed. After filtering, the five best remain, and the guest is shown at most one offer per restaurant. Breadth while searching, restraint when presenting.
- **How the balance was set.** The weighting between meaning and eligibility is a single adjustable setting rather than a fixed rule, and it was chosen by trying a range of values against a set of realistic example requests and keeping the one that gave the best recommendations.

5. The main risk: a perfect-sounding offer that cannot be used

The failure worth guarding against is an offer that reads as an excellent match but is not actually usable: it has expired, it is not valid on the requested day, it requires a larger group, or it belongs to a different

restaurant than the one being recommended. A meaning-based search has no sense of any of that, so left to itself it would happily promise a discount the guest cannot claim. For the guest this is no different from an invented offer, and it fails in the worst possible place: at the table, in front of company.

The design reduces this in four ways.

- **The firm facts are never left to interpretation.** Expiry dates, valid days, minimum party sizes and which restaurant an offer belongs to are checked exactly. An offer that fails any of them is removed before ranking, no matter how good a match it seemed.
- **Enough candidates are gathered up front** that those checks narrow the field without emptying it, so the guest is not quietly left with nothing.
- **Honesty when the catalogue does not apply.** When the agent is working with live restaurant data, our offers will not correspond to the restaurants found. Rather than force a bad match, it presents the closest offers as clearly labelled examples of the kind of thing a partner restaurant might run.
- **The behaviour is checked automatically.** A set of realistic requests, each with the restaurant a good answer should surface, is run as an automatic check, so if a future change quietly degrades the recommendations, it is caught rather than discovered by a guest.

Working agent update

Retrieval is built and running as part of the live conversation, not a prototype on the side. A guest can talk to the concierge, receive a shortlist with genuine offers attached to the right restaurants, and complete a booking, with every offer traceable to a real stored record.

Two other things are working alongside it. Each result can be examined to see why it ranked where it did, which keeps the recommendations explainable rather than mysterious. And the same approach powers the concierge's memory of returning guests: it can recognise someone and recall their preferences, and can also find a guest by description rather than by name.

Next: the human approval step before any booking is confirmed, and the audit trail that records what the agent did and why.